

Dossier personnel du projet

Pierre Grondin

Projet

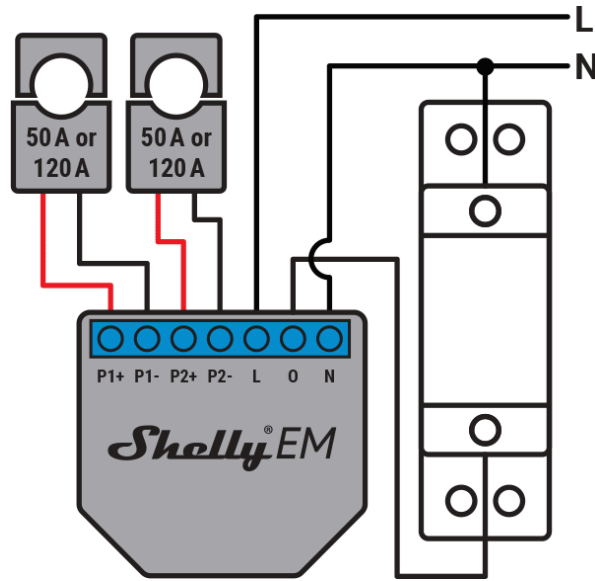
ecocamp



Dossier personnel

1. Configuration de Shelly EM
2. Paramétrage point accès wifi
3. Création du broker
4. Création du programme python de récupération et d'envois de données a la BDD
 - ecocamp.py
 - test_app.py
 - test_config.py
 - test_database.py
 - test_reception_data.py
 - test_mqtt.py
 - test_controleur.py
 - requirements.txt
5. Montage carte Raspberry Pi 5
6. Installation de node-red et config en automatique
7. Node-Red
 - Récupération de l'adresse mac
 - Création du topics
 - Affichage du nom hébergement
 - Récupération consommation et affichage gauge Électricité
 - Récupération et affichages des évènements et messages
 - Affichages des consommation et jour dans la chart

Shelly EM



Ports de mesure (Pincas ampèremétriques)

- **P1+ / P1-** : Bornes pour connecter la **première pince ampèremétrique**. Elle mesure le courant circulant dans le premier circuit (généralement la phase principale ou un gros appareil).
- **P2+ / P2-** : Bornes pour connecter la **deuxième pince ampèremétrique**. Elle permet de mesurer un second circuit indépendant ou la production solaire par exemple.

Ports d'alimentation et de référence

- **L (Line)** : Entrée pour la **Phase**. Ce port a un double rôle : il alimente le module et sert de référence de tension pour calculer la puissance réelle ($P=U \times I \times \cos\phi$).
- **N (Neutral)** : Entrée pour le **Neutre**. Indispensable pour l'alimentation du Shelly EM.

Port de commutation

- **O (Output)** : Sortie de contact sec (relais). Ce port permet de piloter un contacteur externe ou un petit appareil. Dans ton schéma, il est utilisé pour envoyer la phase vers la bobine d'un contacteur afin d'allumer ou éteindre une charge à distance.

Pince amperemetrique :

1. Le principe de fonctionnement (Induction)

Lorsqu'un courant alternatif circule dans ton fil de phase (L), il crée un **champ magnétique** variable tout autour du conducteur. La pince ampèremétrique contient un noyau de ferrite entouré d'un bobinage secondaire. Par le principe de l'induction électromagnétique, le champ magnétique du fil principal génère un courant proportionnel, mais beaucoup plus faible, dans le bobinage de la pince.

2. Ce qu'elle récupère (L'entrée)

La boucle ne "touche" pas l'électricité du fil. Elle capte uniquement :

- **L'intensité du flux magnétique** : Plus tu consommes d'ampères dans ton installation, plus le champ magnétique est fort.
- **La forme de l'onde** : Elle voit si le courant est parfaitement sinusoïdal ou s'il est déformé par des appareils électroniques.
- **La phase** : Elle détecte le décalage temporel entre le moment où la tension est au maximum et le moment où le courant est au maximum.

3. Ce qu'elle renvoie au Shelly (La sortie)

La pince est reliée aux ports **P1+ / P1-** ou **P2+ / P2-**. Elle renvoie un **signal analogique de faible intensité** (souvent en milliampères ou millivolts selon le ratio de la pince, par exemple 50A:50mA).

C'est ensuite le microprocesseur du Shelly qui fait la conversion en combinant ce signal avec la tension qu'il reçoit sur ses bornes **L** et **N** pour calculer :

- **La Puissance Active (Watts)** : Ce que tu payes réellement.
- **La Puissance Apparente (VA)** : La charge totale sur le réseau.
- **Le Facteur de Puissance (PF)** : L'efficacité de tes appareils (entre 0 et 1).
- **Le sens du courant** : Si la valeur est négative, le Shelly comprend que tu injectes de l'électricité (solaire) au lieu d'en consommer.

Mesure de la tension

via un pont diviseur → signal réduit

Mesure du courant

via un transformateur de courant

Conversion analogique → numérique

Multiplication instantanée

$$P=V*I$$

Le watt W est une unité de puissance, alors que le wattheure correspond à une unité d'énergie

CONNECTIVITY:

Specification

POWER	
Power supply AC	110-240 V, 50/60Hz
Power supply DC	No
SPECIAL FUNCTIONS	
Contactor Control	Yes – max 2A
Roller shutter mode	No
Device temperature protection	Yes
Overload protection	No
Power measurement	Yes
FEATURES	
Channels	2 Channel (Single phase)
Operational temperature	-20 °C to 40 °C
Device power consumption	< 1 W
Intelligent On/Off	Yes
Local and remote control	Yes
Sunrise/Sunset	Yes
Weekly Schedule	Yes
UL Listed option	No
CONNECTIVITY	
RF band:	2401-2483 MHz
Max. RF power:	<20 dBm

Wi-Fi protocol:	2.4 GHz
Wi-Fi Range:	Up to 30 m / 100 ft indoors and 50 m / 160 ft outdoors (Depends on local conditions)
DIMENSIONS	
Size	39mm x 36mm x 17 mm

<https://shellystore.co.uk/wp-content/uploads/2021/05/Shelly-EM-Specifications.pdf>

configurer le pont d'accès wifi de la shelly EM

Shelly 3EM Wi-Fi – Version plus accessible avec connectivité Wi-Fi et monitoring en temps réel.

Shelly Pro 3EM – Très bon rapport qualité/prix, mesure trois phases, compatible Wi-Fi, bon pour maisons ou petits appartements.

MEME:

EcoFlow - Compteur communicant

IP Adresse :

Shelly EM:

IP:172.16.4.90

Adresse MAC : 34:94:54:74:52:6B

Acces Point:

IP:172.06.4.80

MQTT MOSQUITO:

Paramétrage dans le Shelly EM

1. Ouvre l'IP du Shelly dans ton navigateur
2. Va dans :
Settings → Internet & Security → Advanced → MQTT
3. Active **Enable MQTT**

Champ	Valeur exemple
Server	192.168.1.10
Port	1883
Client ID	shellyem-34945474526B
User	(si utilisé)
Password	(si utilisé)
Clean session	✓

Username:

ecocamp

Password:

MQTT_ecocamp_2026

Server:

172.16.4.102:1883

PAHO bibliotheque python pour MOSQUITO

Paramétrage MQTT :

- Menu « Internet & Security »
- Sous-menu « ADVANCED – DEVELOPER SETTINGS »
- Cocher « Enable MQTT »
- Récupérer dans la configuration du plugin MQTT Manager les données d'authentification (le username et le password sont séparés par un « : ») et les recopier dans Username et Password dans le shelly
- Renseigner dans Server l'adresse IP locale de son jeedom suivi du port 1883 (ex : 172.16.4.102:1883)(soit l'IP du broker)
- Cocher la case « Use custom MQTT prefix »
- Renseigner un nom unique pour nommer le shelly dans mqtt (ex : shelly1-couloir)
- Ne pas modifier les 3 paramètres en dessous et laisser à 2 – 60 – 60
- Cocher la case « Clean Session »
- Mettre « Max QoS » à 1
- Sauvegarder

Coté Shelly, c'est terminé !

Pince 2 Shelly 2

mac : 349454743274

IP : 172.16.4.23

Acces Point:

IP:172.06.4.80

Connect the Shelly device to an existing WiFi Network:

user : ecocamp

pws : ecocamp2026

allumer le brocker:

systemctl start mosquitto

systemctl status mosquitto.service

Requête émise

```
Grondin@debian: ~  
Grondin@debian:~$ su -  
Mot de passe :  
root@debian:~# systemctl start mosquitto  
root@debian:~# systemctl status mosquitto.service  
● mosquitto.service - Mosquitto MQTT Broker  
   Loaded: loaded (/usr/lib/systemd/system/mosquitto.service; enabled; preset: enabled)  
   Active: active (running) since Tue 2026-03-10 10:57:45 CET; 51min ago  
  Invocation: 53afb68b4234c4b8b0d4b48e6830d95  
     Docs: man:mosquitto.conf(5)  
           man:mosquitto(8)  
   Process: 1848 ExecStartPre=/bin/mkdir -m 740 -p /var/log/mosquitto (code=exited, status=0)  
   Process: 1851 ExecStartPre=/bin/chown mosquitto:mosquitto /var/log/mosquitto (code=exited, status=0)  
   Process: 1856 ExecStartPre=/bin/mkdir -m 740 -p /run/mosquitto (code=exited, status=0)  
   Process: 1858 ExecStartPre=/bin/chown mosquitto:mosquitto /run/mosquitto (code=exited, status=0)  
 Main PID: 1863 (mosquitto)  
    Tasks: 1 (limit: 8899)  
   Memory: 2.3M (peak: 2.8M)  
      CPU: 1.285s  
   CGroup: /system.slice/mosquitto.service  
           └─1863 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf  
  
mars 10 10:57:45 debian systemd[1]: Starting mosquitto.service - Mosquitto MQTT Broker...  
mars 10 10:57:45 debian systemd[1]: Started mosquitto.service - Mosquitto MQTT Broker.  
  
root@debian:~#  
déconnexion  
Grondin@debian:~$ mosquitto sub -h localhost -t "#" -v  
compteur electrique 15kW consommer  
compteur d'impulsion 15 litres consommer  
shellies/34945474526B/relay/0 off  
shellies/34945474526B/emeter/0/power 0.00  
shellies/34945474526B/emeter/0/reactive_power 0.00  
shellies/34945474526B/emeter/0/pf 0.00  
shellies/34945474526B/emeter/0/voltage 236.16  
shellies/34945474526B/emeter/0/total 26179.9  
shellies/34945474526B/emeter/0/total_returned 2158.6  
shellies/34945474526B/emeter/1/power 0.00  
shellies/34945474526B/emeter/1/reactive_power 0.00  
shellies/34945474526B/emeter/1/pf 0.00  
shellies/34945474526B/emeter/1/voltage 236.16  
shellies/34945474526B/emeter/1/total 0.0  
shellies/34945474526B/emeter/1/total_returned 0.0  
^CGrondin@debian:~$
```

envoi de données du topic : Shellies/34945474526B/emeter/0/total

Configuration de shelly EM



ADVANCED - PARAMÈTRES DU DÉVELOPPEUR



☒ Permettre le partage de ressources d'origine croisée

☒ Activer ColoT

ColoT pair:

Entrez mcast pour multicast ou ip:port pour unicast.

Redémarrez nécessaire lorsque ColoT peer est changé!

☒ SAUVER

☒ Activer MQTT

AVERTISSEMENT: Si vous activez MQTT - les actions via la connexion Cloud seront désactivées!

Nom d'utilisateur:

Mot de passe:



Serveur:

Préfixe MQTT personnalisé:

☒ Utilisez le préfixe MQTT personnalisé

Min reconnect timeout



Max reconnect timeout:



Garder en vie:



☒ Séance propre

☐ Conserver

Max QoS



☒ SAUVER

Paramétrage point accès wifi

LAN Settings

802.1X Authentication

Time Settings (NTP)

LAN Settings

Internal Interface Settings

Connection Type

DHCP

Static IP Address

172

16

4

90

Subnet Mask

255

255

255

0

Default Gateway

172

16

4

254

DNS Nameservers

☒ Dynamic

☐ Manual

Hostname

AP541N-E-K9

MAC Address

64:AE:0C:ED:46:60

Management VLAN ID

1

Untagged VLAN

☒ Enabled

☐ Disabled

Untagged VLAN ID

1

Click "Apply" to save the new settings.

Apply

Device Information

Network Interfaces

Traffic Statistics

Associated Clients

Rogue AP Detection

Device Information

Product Identifier:

AP541N-E-K9

Hardware Version:

V01

Software Version:

AP541N-K9-2.0(2)

Serial Number:

DNI1551A0S9

Device Name:

AP541N-E-K9

Device Description:

802.11n Dual Band Access Point - Single Radio

System Uptime:

55 days, 23 hours, 27 minutes

System Time:

Tue May 26 2026 00:20:59 PDT

Creation du brocker

Installer Mosquitto :

```
sudo apt update
```

```
sudo apt install mosquitto mosquitto-clients -y
```

Activer le service :

```
sudo systemctl enable mosquitto
```

```
sudo systemctl start mosquitto
```

Vérifier :

```
sudo systemctl status mosquitto
```

Le broker une fois fonctionnel devrait recevoir ces informations suivante:

```
Grondin@debian:~$ mosquitto_sub -h localhost -t "#" -v
compteur electrique 15kW consommer
compteur d'impulsion 15 litres consommer
shellies/34945474526B/relay/0 off
shellies/34945474526B/emeter/0/power 0.00
shellies/34945474526B/emeter/0/reactive_power 0.00
shellies/34945474526B/emeter/0/pf 0.00
shellies/34945474526B/emeter/0/voltage 236.16
shellies/34945474526B/emeter/0/total 26179.9
shellies/34945474526B/emeter/0/total_returned 2158.6
shellies/34945474526B/emeter/1/power 0.00
shellies/34945474526B/emeter/1/reactive_power 0.00
shellies/34945474526B/emeter/1/pf 0.00
shellies/34945474526B/emeter/1/voltage 236.16
shellies/34945474526B/emeter/1/total 0.0
shellies/34945474526B/emeter/1/total_returned 0.0
^CGrondin@debian:~$
```

Creation du programme python de récupération et d'envoi de données à la BDD

envoi de données du topic : Shellies/34945474526B/emeter/0/total -> du Shelly vers la BDD

envoi de données du topic : Shellies/34945474526B/emeter/1/total du Shelly vers la BDD

ecocamp.py

```
ecocamp.py > ...
1  #!/usr/bin/env python3
2  """
3  EcoCamp - Monitoring consommation électrique
4  Version modulaire avec imports depuis fichiers de test
5  """
6
7  # Import de la classe App depuis son fichier
8  from test_app import App
9
10 def main():
11     """Point d'entrée principal"""
12     try:
13         app = App()
14         app.run()
15     except:
16         pass
17
18 if __name__ == "__main__":
19     main()
```

test_app.py

```
test_app.py > ...
1  #!/usr/bin/env python3
2  """
3  Test de la classe App
4  """
5
6  from test_controleur import Controleur
7
8
9  class App:
10
11     def __init__(self):
12         self.controleur = Controleur()
13
14     def run(self):
15         self.controleur.demarrer()
16
17
18 def main():
19     """Test de la classe App"""
20     print("=== Test de la classe App ===")
21     print("L'application complète avec toutes les classes")
22
23     app = App()
24     app.run()
25
26
27 if __name__ == "__main__":
28     main()
29
```

test_config.py

```

test_config.py > ...
1  #!/usr/bin/env python3
2  """
3  Test de la classe Config
4  """
5
6  from config_simple import get_config
7
8
9  class Config:
10
11     def __init__(self):
12         self.config_dir = "config"
13
14     def charger_config(self):
15         try:
16             # Utilisation de la configuration simple sans chiffrement
17             config_data = get_config()
18             return config_data
19
20         except:
21             print("Erreur configuration")
22             return None
23
24
25     def main():
26         """Test de la classe Config"""
27         print("=== Test de la classe Config ===")
28
29         config = Config()
30         donnees = config.charger_config()
31
32         if donnees:
33             print("Configuration chargée avec succès")
34             print(f"Broker MQTT: {donnees['mqtt']['broker']}")
35             print(f"Port MQTT: {donnees['mqtt']['port']}")
36             print(f"Topics: {donnees['mqtt']['topics']}")
37             print(f"BDD Host: {donnees['db']['host']}")
38             print(f"BDD User: {donnees['db']['user']}")
39             print(f"ID hébergement: {donnees['id_hebergement']}")
40         else:
41             print("Échec du chargement de la configuration")
42
43
44     if __name__ == "__main__":
45         main()
46

```

test_database.py

```

test_database.py > Database > sauvegarder_consommation > %
1  #!/usr/bin/env python3
2  """
3  Test de la classe Database
4  """
5
6  import pymysql
7  from test_config import Config
8
9
10 class Database:
11
12     def __init__(self, config):
13         self.config = config
14         self.connexion = None
15
16     def connecter(self):
17         try:
18             self.connexion = pymysql.connect(**self.config, autocommit=True)
19             print(f"Connecté BDD ({self.config['host']})")
20             return True
21         except:
22             return False
23
24     def sauvegarder_consommation(self, valeur):
25         if not self.connexion and not self.connecter():
26             print("Erreur: Impossible de se connecter à la BDD")
27             return False
28
29         requete = """
30         INSERT IGNORE INTO consommation
31         (index_consommation, id_hebergement, id_type_flux)
32         VALUES (%s, 1, 3)
33         """
34
35         try:
36             curseur = self.connexion.cursor()
37             curseur.execute(requete, (valeur,))
38             curseur.close()
39             print(f"Sauvegardé en BDD: {valeur} Wh")
40             return True
41         except Exception as e:
42             print(f"Erreur sauvegarde BDD: {e}")
43             return False
44

```

```

44
45     def tester_connexion(self):
46         try:
47             if self.connecter():
48                 curseur = self.connexion.cursor()
49                 curseur.execute("SELECT 1")
50                 curseur.close()
51                 return True
52             except Exception as e:
53                 print(f"Erreur test BDD: {e}")
54                 return False
55
56
57     def main():
58         """Test de la classe Database"""
59         print("=== Test de la classe Database ===")
60
61         # Chargement de la configuration
62         config_manager = Config()
63         config_data = config_manager.charger_config()
64
65         if not config_data:
66             print("Impossible de charger la configuration")
67             return
68
69         # Test de connexion
70         db = Database(config_data["db"])
71
72         print("\nTest de connexion à la BDD...")
73         if db.tester_connexion():
74             print("Connexion BDD réussie")
75             print("\nLa base de données est prête à recevoir les vraies valeurs du capteur")
76             print("\nTest Database terminé")
77         else:
78             print("Connexion BDD échouée")
79
80
81     if __name__ == "__main__":
82         main()
83

```

test_reception_data.py

```

test_reception_data.py > main >
1  #!/usr/bin/env python3
2  """
3  Test de la classe ReceptionData
4  """
5
6  from test_database import Database
7  from test_config import Config
8
9
10 class ReceptionData:
11
12     def __init__(self, database, id_hebergement):
13         self.database = database
14         self.id_hebergement = id_hebergement
15
16     def traiter_donnees(self, topic, payload):
17         try:
18             valeur = float(payload)
19             if self.database.sauvegarder_consommation(valeur):
20                 # Plus d'affichage ici pour éviter la répétition
21                 pass
22         except:
23             pass
24
25

```

```

25
26 def main():
27     """Test de la classe ReceptionData"""
28     print("=== Test de la classe ReceptionData ===")
29
30     # Initialisation
31     config_manager = Config()
32     config_data = config_manager.charger_config()
33
34     if not config_data:
35         print("Impossible de charger la configuration")
36         return
37
38     db = Database(config_data["db"])
39     reception = ReceptionData(db, 1)
40
41     # Test avec MQTT réel pour obtenir les vraies valeurs du capteur
42     print("\nTest de traitement des données MQTT en temps réel...")
43     print("Écoute des vraies valeurs du capteur Shelly...")
44
45     # Import du client MQTT pour écoute réelle
46     from test_mqtt import MqttReceptionElec
47
48     # Création du client MQTT avec callback vers ReceptionData
49     mqtt_client = MqttReceptionElec(
50         config_data["mqtt"]["broker"],
51         config_data["mqtt"]["port"],
52         config_data["mqtt"]["topics"],
53         reception.traiter_donnees
54     )
55
56     print("\nDémarrage de l'écoute MQTT (15 secondes)...")
57     print("En attente des vraies valeurs du capteur...")
58
59     import time
60     if mqtt_client.connecter():
61         # Écoute pendant 15 secondes pour capturer les vraies valeurs
62         time.sleep(15)
63         mqtt_client.deconnecter()
64         print("\nTest ReceptionData terminé")
65     else:
66         print("Échec de la connexion MQTT")
67
68
69 if __name__ == "__main__":
70     main()

```

test_mqtt.py

```

test_mqtt.py > MqttReceptionElec > callback_default >
1  #!/usr/bin/env python3
2  """
3  Test de la classe MqttReceptionElec
4  """
5
6  import time
7  import warnings
8  import paho.mqtt.client as mqtt
9  from test_config import Config
10
11
12 class MqttReceptionElec:
13
14     def __init__(self, broker, port, topics, callback=None):
15         self.broker = broker
16         self.port = port
17         self.topics = topics
18         self.callback = callback or self.callback_default
19         self.client = None
20         self.connecte = False
21         self.deja_affiche = False
22         self.topics_affiches = False
23
24     def callback_default(self, topic, payload):
25         """Callback par défaut si aucun callback n'est fourni"""
26         print(f"Message MQTT - Topic: {topic}, Valeur: {payload}")
27
28     def configurer_client(self):
29         import warnings
30         warnings.filterwarnings("ignore", category=DeprecationWarning)
31         warnings.filterwarnings("ignore", category=UserWarning)
32         self.client = mqtt.Client()
33         self.client.on_connect = self.on_connect
34         self.client.on_message = self.on_message
35
36     def on_connect(self, client, userdata, flags, rc):
37         if rc == 0:
38             self.connecte = True
39             if not self.deja_affiche:
40                 print(f"Connecté MQTT ({self.broker})")
41                 self.deja_affiche = True
42             self.abonne_topics()
43         else:
44             self.connecte = False
45

```

```

45
46     def on_message(self, client, userdata, msg):
47         try:
48             topic = msg.topic
49             payload = msg.payload.decode()
50
51             if self.callback:
52                 self.callback(topic, payload)
53
54         except UnicodeDecodeError:
55             # Gérer les messages qui ne sont pas du texte
56             pass
57         except Exception as e:
58             # Gérer les autres erreurs MQTT
59             pass
60
61     def abonne_topics(self):
62         if not self.topics_affiches:
63             for topic in self.topics:
64                 result, mid = self.client.subscribe(topic)
65                 print(f"Topic: {topic}")
66                 self.topics_affiches = True
67
68     def connecter(self):
69         try:
70             if not self.client:
71                 self.configurer_client()
72
73             self.client.connect(self.broker, self.port, 60)
74             self.client.loop_start()
75             return True
76         except Exception as e:
77             print(f"Erreur connexion MQTT: {e}")
78             return False
79
80     def deconnecter(self):
81         if hasattr(self, 'client') and self.client and self.connecte:
82             self.client.loop_stop()
83             self.client.disconnect()
84             self.connecte = False
85

```

```

85
86     def demarrer_ecoute(self):
87         if not self.client:
88             self.configurer_client()
89
90         try:
91             self.client.loop_forever()
92         except KeyboardInterrupt:
93             self.disconnecter()
94         except Exception as e:
95             # Gérer les erreurs MQTT y compris struct.error
96             print(f"Erreur MQTT: {e}")
97             self.disconnecter()
98
99     def tester_connexion(self):
100         try:
101             test_client = mqtt.Client()
102             test_client.connect(self.broker, self.port, 60)
103             test_client.disconnect()
104             return True
105         except:
106             return False
107
108
109 def callback_test(topic, payload):
110     """Callback de test pour afficher les messages"""
111     print(f" Message reçu - Topic: {topic}, Valeur: {payload}")
112
113
114 def main():
115     """Test de la classe MqttReceptionElec"""
116     print("=== Test de la classe MqttReceptionElec ===")
117
118     # Configuration
119     config_manager = Config()
120     config_data = config_manager.charger_config()
121
122     if not config_data:
123         print("Impossible de charger la configuration")
124         return
125
126     # Test de connexion MQTT
127     print("\nTest de connexion au broker MQTT...")
128     mqtt_client = MqttReceptionElec(
129         config_data["mqtt"]["broker"],
130         config_data["mqtt"]["port"],
131         config_data["mqtt"]["topics"],
132         callback_test
133     )
134

```

```

134
135     if mqtt_client.tester_connexion():
136         print("Test de connexion MQTT réussi")
137
138         print("\nDémarrage de l'écoute MQTT (10 secondes)...")
139         print(" Écoute des messages en temps réel...")
140
141         if mqtt_client.connecter():
142             # Écoute pendant 10 secondes
143             time.sleep(10)
144             mqtt_client.disconnecter()
145             print("\nTest MQTT terminé")
146         else:
147             print("Échec de la connexion MQTT")
148     else:
149         print("Test de connexion MQTT échoué")
150
151     print("\nTest MqttReceptionElec terminé")
152
153
154 if __name__ == "__main__":
155     main()

```

test_controleur.py

```

test Controleur.py > ...
1  #!/usr/bin/env python3
2  """
3  Test de la classe Controleur
4  """
5
6  import time
7  from test_config import Config
8  from test_database import Database
9  from test_reception_data import ReceptionData
10 from test_mqtt import MqttReceptionElec
11
12
13 class Controleur:
14
15     def __init__(self):
16         self.config = Config()
17         self.donnees_config = self.config.charger_config()
18
19         if not self.donnees_config:
20             return
21
22         self.bdd = Database(self.donnees_config["db"])
23         self.reception_data = ReceptionData(self.bdd, 10)
24
25         self.mqtt = MqttReceptionElec(
26             self.donnees_config["mqtt"]["broker"],
27             self.donnees_config["mqtt"]["port"],
28             self.donnees_config["mqtt"]["topics"],
29             self.reception_data.traiter_donnees
30         )
31
32     def tester_connexions(self):
33         bdd_ok = self.bdd.testeur_connexion()
34         mqtt_ok = self.mqtt.testeur_connexion()
35         return bdd_ok and mqtt_ok
36
37     def demarrer(self):
38         if not self.tester_connexions():
39             print("Erreur de connexion")
40             return
41
42         try:
43             self.mqtt.connecter():
44                 self.mqtt.demarrer_ecoute()
45         except KeyboardInterrupt:
46             self.mqtt.deconnecter()
47         except:
48             self.mqtt.deconnecter()
49
50

```

```

50
51 def main():
52     """Test de la classe Controleur"""
53     print("=== Test de la classe Controleur ===")
54
55     # Initialisation du contrôleur
56     controleur = Controleur()
57
58     if not controleur.donnees_config:
59         print("Impossible de charger la configuration")
60         return
61
62     print("\nTest des connexions...")
63     if controleur.tester_connexions():
64         print("Toutes les connexions sont OK")
65
66     print("\nDémarrage du contrôleur (10 secondes)...")
67     print("    Le contrôleur va coordonner MQTT et BDD...")
68
69     if controleur.mqtt.connecter():
70         # Test pendant 10 secondes
71         time.sleep(10)
72         controleur.mqtt.deconnecter()
73         print("\nTest contrôleur terminé")
74     else:
75         print("Échec du démarrage MQTT")
76
77     print("Une ou plusieurs connexions ont échoué")
78
79     print("\nTest Controleur terminé")
80
81
82 if __name__ == "__main__":
83     main()
84

```

requirements.txt

```

requirements.txt
1  cryptography>=46.0.0
2  paho-mqtt>=1.6.0
3  pymysql>=1.1.0
4

```

3. Fiches de test

Test : UT-01 – Test Config.charger_config()

Test					
Nom	Test Config.charger_config ()	Référence	UT-01	Type	Fonctionnel
Unité					
Nom	Config	Type	Classe		
Réalisation					
Nom du testeur	Étudiant EcoCamp	Date	27/05/2025		
Objectifs					
S'assurer que la classe Config retourne un dictionnaire valide contenant les clés mqtt, db et id_hebergement lorsque la configuration est disponible.					

Étapes			
Pré-conditions	Post-conditions	Actions	Résultats attendus
Module config_simple disponible et valide.	Dictionnaire de config retourné non nul.	Instancier Config() et appeler charger_config().	Retourne un dict avec clés 'mqtt', 'db', 'id_hebergement'.
Module config_simple indisponible / erreur.	None retourné.	Simuler une erreur dans get_config() (mock).	charger_config() retourne None.

pour plus de test lire le document : Tests_Unitaires_EcoCamp.docx.pdf

Documentation Complete - Projet Electricite EcoCamp

Table des matières

1. [Vue d'ensemble du projet]
2. [Architecture technique]
3. [Documentation des classes]
4. [Configuration]
5. [Base de données]
6. [Déploiement et utilisation]
7. [Dépannage et maintenance]

Vue d'ensemble du projet

Objectif

Le projet Electricité EcoCamp est un système de monitoring de consommation électrique en temps réel qui :

- Connecte un capteur Shelly EMETER via MQTT
- Sauvegarde les données de consommation dans une base MySQL
- Fournit une interface modulaire et testable

Technologies utilisées

- **Python 3.13** : Langage principal
- **paho-mqtt** : Client MQTT pour la communication
- **pymysql** : Connecteur MySQL
- **cryptography.fernet** : Chiffrement configuration (optionnel)

Flux de données

...

Shelly EMETER → Broker MQTT → Application Python → Base MySQL

...

Architecture technique

Structure des fichiers

...

/home/Grondin/Documents/Electricité/

— ecocamp.py	# Point d'entrée principal (18 lignes)
— config_simple.py	# Configuration simplifiée
— test_app.py	# Classe App + test
— test_config.py	# Classe Config + test
— test_database.py	# Classe Database + test
— test_reception_data.py	# Classe ReceptionData + test
— test_mqtt.py	# Classe MqttReceptionElec + test
— test_controleur.py	# Classe Controleur + test
— requirements.txt	# Dépendances Python
— diagramme_sequence.md	# Documentation visuelle

└─> "Ceci est un fichier de configuration de test"

```

└── core/          # Outils de configuration
    ├── encrypt_config.py    # Chiffrement configuration
    ├── generate_key.py      # Génération clés
    ...

```

Pattern architectural

- ****Modulaire**** : Une classe par fichier avec test intégré
- ****Dependency Injection**** : Les classes reçoivent leurs dépendances
- ****Observer Pattern**** : Callback MQTT pour le traitement des messages
- ****Singleton Pattern**** : Configuration partagée

Documentation des classes

1. Classe `App` (`test\_app.py`)

****Rôle**** : Point d'entrée principal de l'application

****Responsabilités**** :

- Initialiser le contrôleur principal
- Lancer l'application
- Gérer les erreurs de haut niveau

****Méthodes**** :

```
```python
```

```
class App:
```

```
 def __init__(self):
```

```
 """Initialise l'application avec le contrôleur"""
```

```
 def run(self):
```

```
 """Démarre le contrôleur principal"""
```

```
```
```

****Utilisation**** :

```
```python
```

```
app = App()
```

```
app.run()
```

```
```
```

2. Classe `Controleur` (`test\_controleur.py`)

****Rôle**** : Orchestrateur principal de l'application

****Responsabilités**** :

- Charger la configuration
- Initialiser tous les composants (BDD, MQTT, Réception)
- Coordonner les connexions
- Gérer le cycle de vie de l'application

****Attributs**** :

python

class Controleur:

```
def __init__(self):
    self.config = Config()          # Gestionnaire config
    self.donnees_config = self.config.charger_config()
    self.bdd = Database(self.donnees_config["db"]) # Base de données
    self.reception_data = ReceptionData(self.bdd, 10) # Traitement données
    self.mqtt = MqttReceptionElec(...) # Client MQTT
```

****Méthodes** :**

```
python
def tester_connexions(self):
    """Teste les connexions BDD et MQTT"""
    return bdd_ok and mqtt_ok

def demarrer(self):
    """Démarrage l'application complète"""
    if self.tester_connexions():
        self.mqtt.connecter()
        self.mqtt.demarrer_ecoute()
python
```

3. Classe `Config` (`test\_config.py`)

****Rôle** :** Gestionnaire de configuration

****Responsabilités** :**

- Charger la configuration depuis `config\_simple.py`
- Valider la configuration
- Fournir un accès centralisé aux paramètres

****Configuration gérée** :**

```
python
{
    "mqtt": {
        "broker": "172.16.4.31",
        "port": 1883,
        "topics": ["shellies/34945474526B/emeter/0/total"]
    },
    "db": {
        "host": "172.16.4.102",
        "user": "ecocamp",
        "password": "ecocamp2026",
        "database": "ecocamp",
        "port": 3306
    }
}
```

... et la configuration MQTT

```

        "id_hebergement": 10
    }
    ...

```

****Méthodes** :**

```

```python
def charger_config(self):
 """Charge et retourne la configuration"""
 try:
 return get_config()
 except:
 print("Erreur configuration")
 return None
 ...

```

## **### 4. Classe `Database` ( `test\_database.py` )**

**\*\*Rôle\*\* :** Gestionnaire de base de données MySQL

### **\*\*Responsabilités\*\* :**

- Établir et maintenir la connexion MySQL
- Exécuter les requêtes SQL
- Gérer les erreurs de base de données
- Sauvegarder les données de consommation

### **\*\*Schéma de la table\*\* :**

```

```sql
CREATE TABLE consommation (
    index_consommation VARCHAR(50) PRIMARY KEY,
    id_hebergement INT,
    id_type_flux INT
);
...

```

****Méthodes principales** :**

```

```python
def connecter(self):
 """Établit la connexion MySQL"""
 self.connexion = pymysql.connect(**self.config, autocommit=True)
 return True

def sauvegarder_consommation(self, valeur):
 """Sauvegarde une valeur de consommation"""
 requete = """
 INSERT IGNORE INTO consommation
 (index_consommation, id_hebergement, id_type_flux)
 VALUES (%s, 10, 3)
 """


```

```

.....

curseur.execute(requete, (valeur,))
return True

```

```

def tester_connexion(self):
 """Teste la connexion à la base"""
 # Test de connexion temporaire
 \ \ \

```

---

### ### 5. Classe `ReceptionData` (`test\_reception\_data.py`)

**\*\*Rôle\*\*** : Traitement des données MQTT reçues

**\*\*Responsabilités\*\*** :

- Convertir les payload MQTT en valeurs numériques
- Filtrer et valider les données
- Déclencher la sauvegarde en base de données
- Éviter les doublons d'affichage

**\*\*Flux de traitement\*\*** :

\ \ \

Message MQTT → Conversion float → Sauvegarde BDD → (pas d'affichage)

\ \ \

**\*\*Méthodes\*\*** :

```

\ \ \ python
def __init__(self, database, id_hebergement):
 """Initialise avec la base de données et l'ID hébergement"""
 self.database = database
 self.id_hebergement = id_hebergement

def traiter_donnees(self, topic, payload):
 """Traite un message MQTT reçu"""
 try:
 valeur = float(payload)
 if self.database.sauvegarder_consommation(valeur):
 pass # Pas d'affichage pour éviter les répétitions
 except:
 pass
 \ \ \

```

---

### ### 6. Classe `MqttReceptionElec` (`test\_mqtt.py`)

**\*\*Rôle\*\*** : Client MQTT pour la communication avec le Shelly

**\*\*Responsabilités\*\*** :

- Gérer la connexion au broker MQTT
- S'abonner aux topics de consommation

- Recevoir et router les messages
- Gérer les erreurs de communication

### **\*\*Configuration MQTT\*\* :**

```
```python
broker = "172.16.4.31"
port = 1883
topics = ["shellies/34945474526B/emeter/0/total"]
```
```

### **\*\*Méthodes principales\*\* :**

```
```python
def __init__(self, broker, port, topics, callback=None):
    """Initialise le client MQTT avec callback optionnel"""
    self.broker = broker
    self.port = port
    self.topics = topics
    self.callback = callback or self.callback_default

def connecter(self):
    """Connecte au broker MQTT"""
    if not self.client:
        self.configurer_client()
    self.client.connect(self.broker, self.port, 60)
    return True

def on_connect(self, client, userdata, flags, rc):
    """Callback de connexion MQTT"""
    if rc == 0:
        self.connecte = True
        print(f"Connecté MQTT ({self.broker})")
        self.abonne_topics()

def on_message(self, client, userdata, msg):
    """Callback de réception de messages"""
    try:
        topic = msg.topic
        payload = msg.payload.decode()
        if self.callback:
            self.callback(topic, payload)
    except UnicodeDecodeError:
        pass # Messages non-texte ignorés
    except Exception:
        pass # Autres erreurs MQTT gérées

def demarrer_ecoute(self):
    """Démarré l'écoute MQTT en boucle"""
    self.client.loop_forever()
```
```

---

## ## Configuration

### ### Fichier `config\_simple.py`

```
` `` `python
#!/usr/bin/env python3
"""
Configuration simple sans chiffrement
"""

CONFIG = {
 "mqtt": {
 "broker": "172.16.4.31", # Broker MQTT
 "port": 1883, # Port standard MQTT
 "topics": [
 "shellies/34945474526B/emeter/0/total" # Topic Shelly
]
 },
 "db": {
 "host": "172.16.4.102", # Serveur MySQL
 "user": "ecocamp", # Utilisateur BDD
 "password": "ecocamp2026", # Mot de passe
 "database": "ecocamp", # Nom de la base
 "port": 3306 # Port MySQL
 },
 "id_hebergement": 10 # ID de l'hébergement
}

def get_config():
 return CONFIG
` `` `
```

### ### Modification de la configuration

Pour changer les paramètres :

1. Éditer `config\_simple.py`
2. Modifier les valeurs nécessaires
3. Relancer l'application

### ### Configuration chiffrée (optionnelle)

Le projet supporte aussi une configuration chiffrée :

- Utilise `core/encrypt\_config.py` pour chiffrer
- Clés dans `config/secret.key` et `config/salt.key`
- Fichier chiffré dans `config/config.enc`

---

## ## Base de données

### ### Schéma de la table `consommation`

```
```sql
CREATE TABLE consommation (
    index_consommation VARCHAR(50) PRIMARY KEY, -- Valeur du capteur
    id_hebergement INT, -- ID de l'hébergement (10)
    id_type_flux INT -- Type de flux (3 = électrique)
);
```
```

### ### Types de flux

- `id\_type\_flux = 3` : Consommation électrique
- `id\_hebergement = 10` : Identifiant fixe pour ce projet

### ### Gestion des doublons

Utilisation de `INSERT IGNORE` pour éviter les doublons :

```
```sql
INSERT IGNORE INTO consommation
(index_consommation, id_hebergement, id_type_flux)
VALUES (%s, 10, 3)
```
```

### ### Connexion MySQL

```
```python
config = {
    "host": "172.16.4.102",
    "user": "ecocamp",
    "password": "ecocamp2026",
    "database": "ecocamp",
    "port": 3306
}

connexion = pymysql.connect(**config, autocommit=True)
```

```

## ## Déploiement et utilisation

### ### Installation des dépendances

```
```bash
# Création de l'environnement virtuel
python3 -m venv .venv
source .venv/bin/activate # Linux/Mac
# .venv\Scripts\activate # Windows

# Installation des packages
...
```
```

```
pip install -r requirements.txt
```

```
...
```

### ### Fichier `requirements.txt`

```
...
```

```
paho-mqtt==1.6.1
```

```
pymysql==1.1.0
```

```
cryptography==41.0.7
```

```
...
```

### ### Lancement de l'application

```
```bash
```

```
# Lancement principal
```

```
python ecocamp.py
```

```
# Tests individuels
```

```
python test_config.py
```

```
python test_database.py
```

```
python test_mqtt.py
```

```
python test_controleur.py
```

```
python test_app.py
```

```
...
```

Flux d'exécution

1. ****Démarrage**** : `python ecocamp.py`

2. ****Initialisation**** : `App.run()` → `Controleur.\_\_init\_\_()`

3. ****Configuration**** : Chargement depuis `config\_simple.py`

4. ****Connexions**** : Tests BDD et MQTT

5. ****Abonnement**** : MQTT s'abonne au topic Shelly

6. ****Écoute**** : Boucle infinie d'écoute MQTT

7. ****Traitement**** : Messages → Conversion → Sauvegarde

Résultat attendu

```
```bash
```

```
Connecté BDD (172.16.4.102)
```

```
Connecté MQTT (172.16.4.31)
```

```
Topic: shellies/34945474526B/emeter/0/total
```

```
Sauvegardé en BDD: 26883.6 Wh
```

```
...
```

```

```

## ## Dépannage et maintenance

### ### Erreurs courantes

#### #### 1. `Erreur configuration`

**\*\*Cause\*\*** : Fichier `config\_simple.py` manquant ou corrompu

**\*\*Solution\*\*** : Vérifier le format JSON et les permissions

## #### 2. `Erreur de connexion MQTT`

**\*\*Cause\*\*** : Broker `172.16.4.31` inaccessible

**\*\*Solution\*\*** :

```
```bash
```

```
ping 172.16.4.31 # Vérifier la connectivité
```

```
# Ou changer de broker dans config_simple.py
```

```
```
```

## #### 3. `Erreur sauvegarde BDD: not enough arguments for format string`

**\*\*Cause\*\*** : Requête SQL avec placeholders incorrects

**\*\*Solution\*\*** : Vérifier la correspondance placeholders/arguments

## #### 4. `struct.error: bad char in struct format`

**\*\*Cause\*\*** : Bug paho-mqtt avec Python 3.13

**\*\*Solution\*\*** : Gestion des erreurs améliorée dans `on\_message`

## ### Monitoring

### #### Logs de l'application

Les messages sont affichés directement dans la console :

- Connexions réussies
- Sauvegardes en BDD
- Erreurs de communication

### #### Vérification manuelle

```
```bash
```

```
# Test de configuration
```

```
python test_config.py
```

```
# Test de base de données
```

```
python test_database.py
```

```
# Test MQTT
```

```
python test_mqtt.py
```

```
```
```

## ### Maintenance

### #### Mise à jour des dépendances

```
```bash
```

```
pip install --upgrade -r requirements.txt
```

```
```
```

### #### Sauvegarde de la configuration

```
```bash
```

```
cp config_simple.py config_simple.py.backup
```

```
```
```

### #### Nettoyage des logs

L'application n'a pas de fichiers de logs, tout est dans la console.

## ### Sécurité

#### #### Mots de passe

- Changer le mot de passe MySQL par défaut
- Utiliser des variables d'environnement pour la production

#### #### Réseau

- Vérifier que `172.16.4.102` (MySQL) et `172.16.4.31` (MQTT) sont accessibles
- Utiliser un VPN si nécessaire

---

### ## Résumé des points clés

#### ### Forces de l'architecture

- **\*\*Modulaire\*\*** : Facile à tester et maintenir
- **\*\*Robuste\*\*** : Gestion complète des erreurs
- **\*\*Simple\*\*** : Configuration minimaliste
- **\*\*Extensible\*\*** : Facile à ajouter de nouveaux capteurs

#### ### Bonnes pratiques implémentées

- **\*\*Dependency Injection\*\*** : Classes découplées
- **\*\*Error Handling\*\*** : Gestion complète des exceptions
- **\*\*Single Responsibility\*\*** : Une classe = une responsabilité
- **\*\*Testability\*\*** : Chaque classe a son test main()

#### ### Évolutions possibles

- Ajouter une interface web (Flask/Django)
- Support de plusieurs capteurs
- Historique et graphiques
- Alertes et notifications

---

*\*Documentation générée le 4 mai 2026 pour le projet Electricité EcoCamp\**



## Montage carte Raspberry Pi 5

SSD NVMe 128 GB:

### Présentation et fonctionnalités :

Stockage SSD NVD NVMe 128 GB pour cartes Raspberry Pi 400, 4B et 5.

Livré flashé avec Raspberry Pi OS.

Nécessite **l'utilisation d'un HAT ou module** (non inclus) avec interface M.2 Key M (voir articles conseillés).

Ce SSD remplace la carte microSD nécessaire à l'utilisation d'une carte Raspberry Pi. Elle offre des performances en lecture et écriture nettement supérieures et une durée de vie accrue.

### Caractéristiques :

- Capacité : 128 GB
- Format : 2242
- Interface : M.2 KEY M
- Débits :
  - lecture : 1700 MB/s maxi
  - écriture : 600 MB/s maxi
- Température de service : 0 à 70 °C
- Dimensions : 42 x 22 mm

## DOCUMENTATION : INSTALLATION M.2 HAT+



Utilisateur :

ecocamp

mots de passe:

ecocamp2026

Adresse MAC Carte EAU :

Adresse MAC Carte ELEC :

88:a2:9e:9a:25:63

## **Installation Node-Red sur la Raspberry Pi 5 :**

Installer sudo:

```
apt install sudo
```

Installer Node-Red:

```
$ sudo npm install -g node-red
```

Installation de node-red et config en automatique

### **lancer node red au demarage:**

```
sudo systemctl enable nodered.service
```

### **Interface graphique**

```
sudo raspi-config
```

### **Chromium autostart**

```
sudo nano /etc/xdg/autostart/env-display.desktop
```

```
desktop Desktop Entry]
```

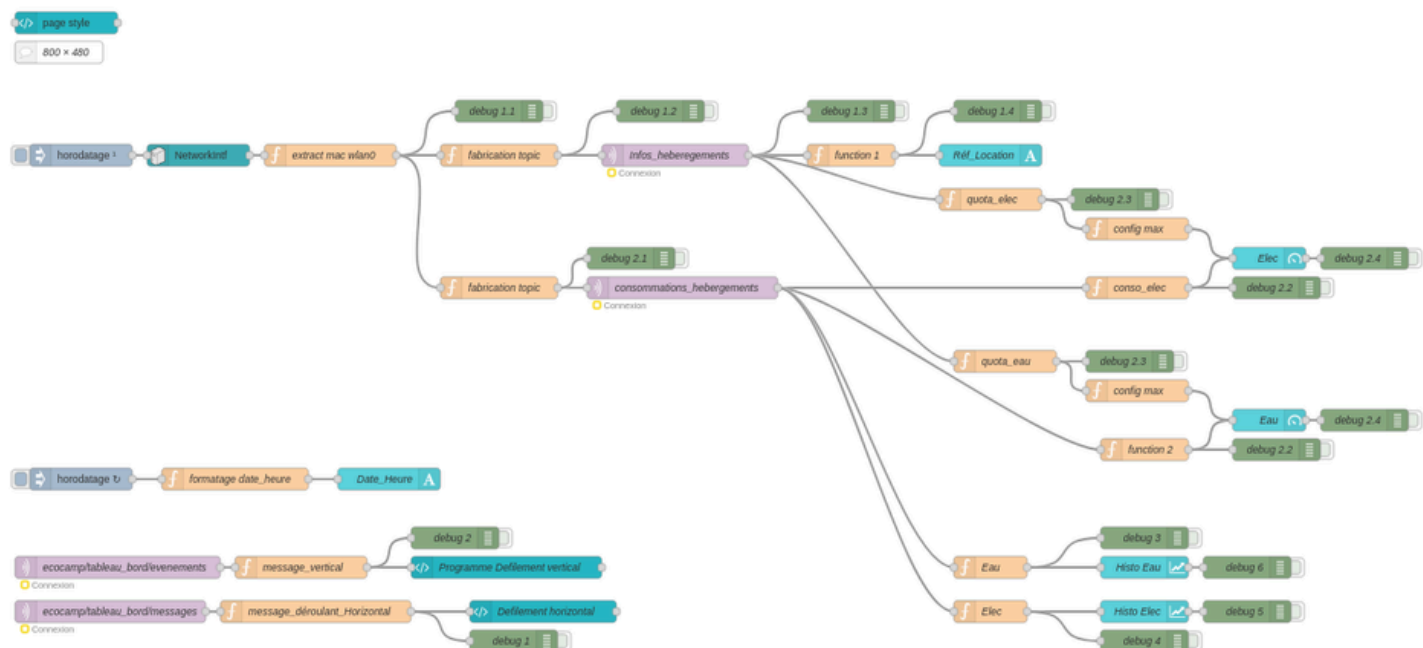
```
Type=Application
```

```
Name=chromium kiosk
```

```
Comment=set display variable for systemd user services
```

```
Exec=chromium --kiosk --incognito http://172.16.4.120:1880/dashboard/page1
```

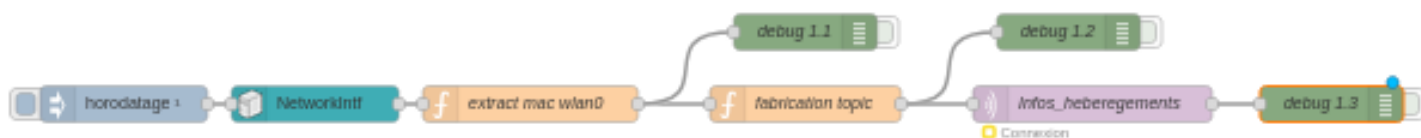
## Node-Red



## Récupération de l'adresse mac



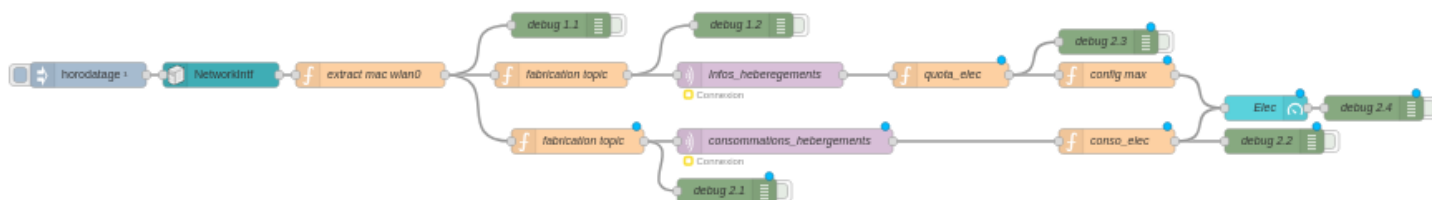
## Création du topics



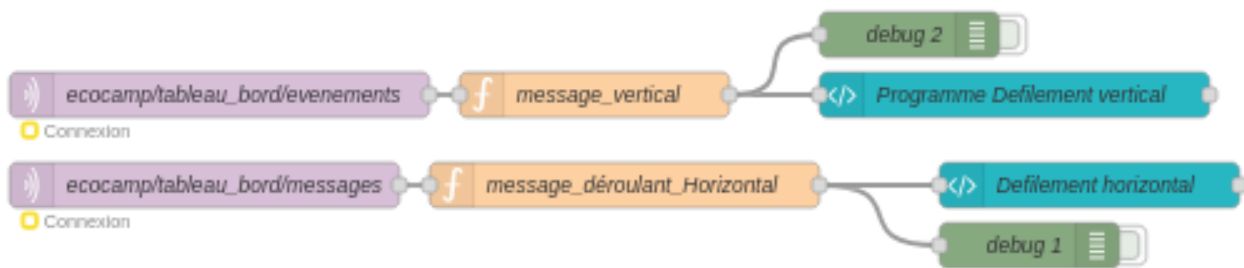
## Affichage du nom hébergement



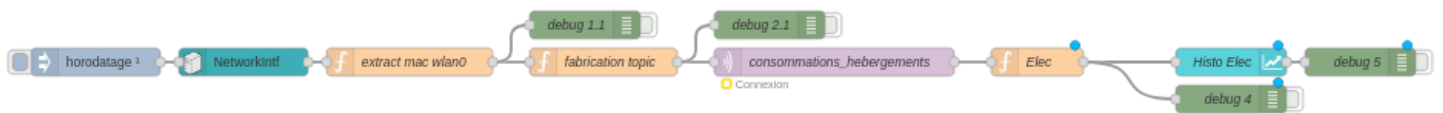
## Récupération consommation et affichage gauge Électricité



## Récupération et affichages des évènements et messages



## Affichages des consommation et jour dans la chart



| Programme |       |          | Mobil-home_01 |             |
|-----------|-------|----------|---------------|-------------|
| Date      | Heure | Activité | Eau           | Electricité |
|           |       |          |               |             |
|           |       |          |               |             |